# ai bot

©syui

# Table of Contents


1. ai bot 1.1
2. @yui.syui.ai 1.2
    1. 原作 1.2.1
    2. 画像 1.2.2
3. 技術 1.3
    1. archlinux 1.3.1
    2. rust 1.3.2
    3. bluesky 1.3.3
4. 漫画 1.4
    1. 0 1.4.1
    2. 1 1.4.2
    3. 2 1.4.3
    4. 3 1.4.4
    5. fix 1.4.5
5. end 1.5

# ai bot

# @yui.syui.ai

# 原作

`bluesky`には、mention(メンション)するとカードが貰えるという不思議なbotが住んでいます。

今回は、このbotがどういった経緯や技術で誕生したのかを紹介します。

このbotの正式名称は`aibot`。blueskyに常駐する一つのアカウントです。

handleは`@yui.syui.ai`、nameは`ai`です。

- handle : @yui.syui.ai
- name : `ai`

アイコンは輪っかをつけた女の子。アイちゃんとか、ユイちゃんと呼ばれています。

「yui」という作品のアイとキャラクターがモデルになっています。

ちなみに、原作では性別不詳です。明かされたことがないのでわかりません。

見た目が女の子なので、私は彼女と呼んでいます。

まず、中身の技術を紹介します。

最後におまけの漫画があります。

# 画像

# 技術

早速、aibotの中身を見ていきましょう。

aibotはarchlinux server(サーバー)、rustでcompile(コンパル)されたbinary(バイナリ)から動いています。

つまり、archlinux+rustが中身と言えます。

簡単にarchlinuxとrustの説明を行います。

archlinuxは、linuxディストリビューションの一つ、osのことです。linux kernel(リナックス・カーネル)で動いているので、arch linuxと言います。

archlinuxは、シンプル必要最小限のosというイメージです。source(ソース)をbuild(ビルド)することもできますし、そういう仕組がAURにありますが、buildされたbinaryをdownloadすることもできます。ただし、パッケージ数はそれほど多くはありません。

rustは、様々なプログラミング言語の中で非常に難易度が高い言語と言われています。一度構築すると安定して動作するけど、動かすまでに時間がかかるというイメージです。

この章では、簡単にarchlinuxとrustの始め方を紹介します。

**用語の解説**

| 用語 | 意味 |
|---|---|
| server(サーバー) | 大体は応答に応じる外部パソコンのこと |
| linux(リナックス) | 大体はlinux kernelかosのこと。archのほかにgentoo, ubuntuなどがある |
| build(ビルド), compile(コンパイル) | ソースコードをそのコンピュータで実行できる形式に変換すること |
| binary(バイナリ) | buildしてできた実行形式のファイルのこと。binと略される |
| deploy(デプロイ) | 大体はbuildしたファイルをserverに展開すること |
| source(ソース) | 人間が読めるプログラムコードのこと。srcと略される |
| package(パッケージ) | アプリのこと。pkgと略される |
| rust(ラスト) | プログラム言語の一つ。cの代わりと目されている。linuxもcからrustに書き換えられている最中 |

# archlinux

### archlinux

まず、archlinuxのisoをdownloadします。ここでは`jaist`, `tsukuba`のftpを紹介します。どちらも日本にあるserverなので速度的には有利だと思われます。

http://ftp.jaist.ac.jp/pub/Linux/ArchLinux/iso/latest/

http://ftp.tsukuba.wide.ad.jp/Linux/archlinux/iso/latest/

archlinuxの公式ページは`archlinux.org`と`archlinux.jp`があります。

en : https://archlinux.org

ja : https://archlinux.jp

`.org`が本家、`.jp`が日本コミュニティです。

`iso`をdownloadできたら、それをusbに置きます。usbは2Gもあれば十分でしょう。

```
$ sudo dd if=./archlinux.iso of=/dev/sdx bs=4M
```

これは色々なやり方があります。

```
# cat path/to/archlinux-version-x86_64.iso > /dev/disk/by-id/usb-My_flash_drive
# cp path/to/archlinux-version-x86_64.iso /dev/disk/by-id/usb-My_flash_drive
```

まずusbがどこに接続されているかを調べます。そこを指定して、`dd`や`cat`のコマンドを実行します。現在のosがインストールされているdiskを指定しないよう注意してください。この例では`sda`を指定してはいけません。`sdb`を指定します。

```
$ sudo fdisk -l
sda ... 100G
sdb ... 2G

$ sudo dd if=~/Download/archlinux-version-x86_64.iso of=/dev/sdb bs=4M
```

なお、`mac`では`diskutil`が必要です。

出来上がったusbをパソコンに挿して、usb-boot(usbからimgを起動)して、archlinuxをインストールします。

```
$ archinstall
```

archlinuxは、非常にマニアックなosですが、最先端のエンジニアに人気があります。

最近でいうと、mac-m1にインストールできるasahi linuxというものがあって、archlinuxが採用されています。

また、steam deckというゲーム機が販売されていますが、その`steam os`は、archlinuxをベースにしています。

# rust

### rust

rustはそれぞれのpackage manager(パッケージ・マネージャー)でインストールしてください。archlinuxならpacmanを使用します。

```
pacman -S rust
```

```
$ cargo version
cargo 1.70.0
```

まず、rustでプログラムの雛形を作ります。

```
mkdir -p ~/rust
cd ~/rust
cargo init
```

```
.
├── Cargo.toml
└── src
    └── main.rs
```

これらのファイルは自分で作成してもいいし、initで作成してもいいです。

Cargo.toml

```
[package]
name = "rust"
version = "0.1.0"
edition = "2021"

# See more keys and their definitions at https://doc.rust-
lang.org/cargo/reference/manifest.html

[dependencies]
```

src/main.rs

```
fn main() {
    println!("Hello, world!");
}
```

ちなみに、コンピュータ上ではディレクトリもファイルなのです。この話は面倒なので省略します。

### editor

次にプログラムの中身を確認してみます。

確認するには、editor(エディタ)を使用します。私はvimを使いますが、visual studioがオススメかな。

```
brew install vim
vim src/main.rs
```

src/main.rs

```
fn main() {
    println!("Hello, world!");
}
```

これは`hello world!`という文字列を出力するプログラムです。

**build**

このsrcを`build`して`binary`、つまり、アプリ本体に変換することで、そのパソコンで実行できるようになります。

```
cargo build
```

target/debug/rust

```
target
└── debug
    ├── rust ← このファイルがbinary
    └── rust.d
```

rustはワンバイナリと言って、コンパイル結果が単一ファイルなので、非常に良い言語です。

```
$ ./target/debug/rust
Hello, world!
```

# bluesky

### bluesky

`bluesky`では[rate limit]の変更がありました。

この変更によってbotなどが制限を受けやすくなります。

ここでは、実際にblueskyの負荷を減らし、botが制限されることを回避する一例を紹介します。

具体的には、`createSession`を減らし、代わりに`refreshSession`を使用するようにします。

createSession

- 30/5 min
- 300/day

### refresh token

今までは`createSession`で都度、tokenを取得し、新しいsessionを作成してきました。

`create`は新しいパスワードを作るイメージ、`refresh`はパスワードを更新するイメージです。これらには有効期限が付いています。有効期限付きのパスワードをtokenと呼びます。

`createSession`でapiを呼び出す都度、新しいtokenを発行すれば、たしかに確実ですが、有効なtokenがいくつもできてしまいます。これは、危険でもあり、無駄でもあります。ですから、`refreshSession`で一つのパスワードを更新して使い回す方法が用意されています。`refresh-token`とも呼ばれます。

基本的には、`com.atproto.server.refreshSession`にtokenとして`refreshJwt`を指定します。

```
handle=xxx.bsky.social
pass=xxx
curl -sL -X POST -H "Content-Type: application/json" \
         -d "
{\"identifier\":\"$handle\",\"password\":\"$pass\"}" \

https://bsky.social/xrpc/com.atproto.server.createSession \
         | jq -r .refreshJwt

refresh=`!!`

curl -sL -X POST -H "Content-Type: application/json" -H
"Authorization: Bearer $refresh"
https://bsky.social/xrpc/com.atproto.server.refreshSession
```

例えば、これを`rust`で書いてみます。

```
├── Cargo.toml
└── src
    ├── main.rs
    └── refresh.rs
```

Cargo.toml

```
[package]
name = "ai"
version = "0.1.0"
edition = "2021"

[dependencies]
seahorse = "*"
reqwest = { version = "*", features = ["blocking", "json"] }
tokio = { version = "1", features = ["full"] }
```

src/refresh.rs

```
extern crate reqwest;

pub async fn post_request() -> String {

    let refresh = "xxxxxxxxxxxxxxxxxxxxxxxxxxx";

    let url =
"https://bsky.social/xrpc/com.atproto.server.refreshSession"
;

    let client = reqwest::Client::new();
    let res = client
        .post(url)
        .header("Authorization", "Bearer ".to_owned() +
&refresh)
        .send()
        .await
        .unwrap()
        .text()
        .await
        .unwrap();

    return res
}
```

src/main.rs

```
pub mod refresh;

use seahorse::{App, Command, Context};
use std::env;

fn main() {
    let args: Vec<String> = env::args().collect();
    let app = App::new(env!("CARGO_PKG_NAME"))
        .command(
            Command::new("refresh")
            .alias("r")
            .action(c_refresh),
            )
        ;
    app.run(args);
}

fn refresh() {
    let h = async {
        let str = refresh::post_request().await;
        println!("{}",str);
    };
    let res =
tokio::runtime::Runtime::new().unwrap().block_on(h);
    return res
}
```

```
fn c_refresh(_c: &Context) {
    refresh();
}
```

cargo buildして、出来上がったbinaryを実行します。

```
$ cargo build

$ ./target/debug/ai r
{"did":"did:plc:kszd56dmyv2phi7ai4apiu4a","handle":"syui.bsk
y.social","accessJwt":"","refreshJwt":""}
```

# 漫画

# 0

# 1

# 2

# 3

# fix

## 下書き

アイはガララ星、上空にある月の家にて、本の下からクスクスと笑った。

ドライはそれに気づき訝しげな表情。

「あれ、その本、ものすごーくつまんないのになんで笑ってるのさ？」

「うん？ちょっと思い出して…クスクス」

アイは、昔、ゼンに聞いた話を思い出していた。

カードの話だ。

「なになに、気になるんだけどっ！」

「ゼンのおじいちゃんが昔、集めると願い事を叶えるカードの話をしたんだ」

「え！願い事を叶えるカードだって！！たしか、そのじいさん、ゴーストの人だっけ、今どこにいるの？」

「せっかくゴーストになったから地球のコアの様子を見てくるって」

「…つまり、そのじいさんは地球の地底に住み着いてるわけか」

「う、うん。そうだと思う」

「.....」

ドライ、考え込む表情。

「そういえば僕、地球に行く用事を思い出した。行ってくる！」

「え？」

さっそく地球に向かうドライ。目指すは地底にゴーストとして住み着いているゼンというじいさんだ。

ドライは、そのゼンからカードのことを聞き出し、億万長者になる願いを叶えてもらうつもりだった。

アイはというと、珍しくラボに入り、何かを作り始めるのだった。

ドライはあちこち聞き周り、やっとのことで地底のマグマ湖の近くでゴーストを見つけた。きっとあれに違いない。

「？」

「はて、ドラゴンか？わしになにか用かな」

「あ、ゼンさんですか。話が早くて助かるなあ」

「早速ですが、おじいさん、カードをください」

「！！？」

「か、かーど？」

「はい、カード」

「なんのことやらさっぱり…まてよ、お主、まさか天空の島のあれのことを言っとるんじゃなかろうな」

「て、天空？カードって天空にあるの？」

「い、いや、天空には…ない」

「！！！」

「じゃあどこにあるのさ？」

「…あれは…ないよ。存在しない」

「そ、そんなあ…こ、こんな地底にわざわざ来たってのに、じいさん、そりゃないよ…」

「そもそもカードが天空島にあったんなら、なんでこんな地の底をうろちょろしているのさ」

ドライは怒って言葉を続ける。

「じいさ…いや、ゼン、おまえがはじめた物語だろ？」

ゼンは口を半開きにし、唖然とした表情でドラゴンを見た。

「…お、お主が言っとることは、何一つよくわからんが」

「もういい！役に立たないゴーストだなっ」

ドラゴンはそう言ってさっそうと飛び去る。

残された老人は呆然とそれを見ていた。

やがて頭をフリフリしてつぶやく。

「あれは、アイが天空の島を滅ぼした時、何もなかったと言っておったからのう…」

老人は、天空に浮かぶ廃墟と少女の姿を思い浮かべながらそういった。

ドラゴンが帰ってみると、床にはカードが散らばっていた。

「わあ、カードだ。まさかこれ！天空島のやつかい？」

「え？ちがうけど」

「そ、そんなあ…じゃあこのゴミは一体何なんだよ」

「アイが作ったの」

「作って何すんのさ」

「宇宙ショップに出すんだ」

「そんなもん売れるわけ無いだろ！疲れた、僕はもう寝るよ」

アイは色々開発していたようだが、数カ月後もカードはやっぱり売れなかった。

ただ、一部のニッチな人たちがやっており、今度大会が開かれるらしい。

アイも参加することにした。

　ここから漫画を書きやすいよう文章省略

大会当日、フロアがあって、アイはキョロキョロ。ドライもついてきた。ドライも少しだけならカードを持っている。

アイは、ブルドックと対戦。結果ボロ負けする。

お嬢さん、そんなカードで大会に出ようとしたのかい？

ん？ドライはカードを見る。アイ:cp13で、ブルドック:cp415

ドライ(ぼくより弱いじゃないか！！)

アイ、怒りに任せて変身しようとする。が、ブルドックは眠りの魔法をアイに放つ。

アイ、眠りに落ちる。

ドライ、どうしたのアイ！

ブルドック、お嬢さん、短気は良くない。私は眠りの魔術にかけては一流なんだよ、騒がしい外野は黙って...

ここでドライを見る。どでかいドラゴンだ。

ドライがアイを気にかけている間、ブルドック、いなくなる。

数時間後、アイが目覚める。ヒューという風の音。ドラゴンの背中のうえ、夜空の海上を飛んでいた。

ドライ、あ、起きた。アイは眠ると重すぎるから大変だったんだよ。

アイ、カードを見つめる。そして、

「アイ、もうやめる」とカードを空中に放り投げた

ドライ、あ、もったいない！

翌日、ドライはカードを拾いに行く。そして、宇宙ショップへ。

ドライ「これ売りたいんだけど...やっぱダメかな？」

少々お待ちください。ヒソヒソ、こ、これはどこで！？

ドライ、え、海の上だけど。

ちょっと再チェックしてみますのでお待ちを。店員カード持っておくにひっこむ。

全部で3億アムになります。

「え？えええええ！？な、なんで！？」

「このカードは初回発行のものであることがわかりました。そのため市場でも値が跳ね上がっており...」

その後、ドライはニコニコ顔で帰ってくる。

「？なにかいいことでもあったの？」

ドライ、そっぽを向いて、なんもないよ、という。

**補足**

| title | var |
| --- | --- |
| 天空島 | ゼンはアイが天空島を滅ぼしたというが、ここの描写ははっきりとさせないようにする。単なる絵で表現、アイの背中と廃墟化した天空の塔を映すだけ。ここは、もともと古代兵器が置いてあった場所。コンピュータが管理していた |
| ジョーの瞳 | ブルドック、ジョーの片目は、天空島の逸話であるカードの模様をしている。話が続く場合の伏線 |
| カードの童話 | 天空の島に伝わるカードの童話は、大体がゼンが言った通り |

# end

```
@yui.syui.ai /card g15
```